

What We Are Building

A plain-language description of the whole project: what it is, who uses it, what it will and will not do, and how we will know whether it worked.

Written without abbreviations or technical shorthand on purpose. If any sentence here needs a computing degree to read, that is a fault in this document, not in the reader. Every term is explained the first time it appears, and there is a glossary at the end.

01

The project in one paragraph

When someone moves money — sends a transfer, taps a card, uses a banking app — a bank has a few seconds to decide whether that looks like the real customer or somebody who has stolen their details. **Risk Radar watches that stream of payments, works out which ones look wrong, and puts the suspicious ones in front of a human being to investigate.** It does not stop payments. It advises a person, and that person decides.

02

Who uses it

Four kinds of people, with deliberately different powers.

PERSON	WHAT THEY DO ALL DAY	WHAT THEY CANNOT DO
Fraud analyst	Works through suspicious payments one at a time. Reads the evidence, calls the customer where needed, and records what turned out to be true.	Cannot close a case. Somebody else signs it off.
Fraud operations lead	Runs the team. Signs off the analysts' work and watches whether the team is keeping up.	Cannot change how the detection works.
Information security analyst	Handles cases where somebody's account appears to have been broken into, rather than cases about money lost.	Cannot declare something to be fraud. That decision belongs to the fraud team.
Administrator	Adjusts the settings — which warning signs are switched on, how sensitive the system is, which version of the detection is running.	Cannot open, read or decide any case at all.

WHY THE ADMINISTRATOR IS LOCKED OUT

Somebody who can make the system less sensitive, and then look at the cases it failed to raise, could quietly hide their own mistakes. So they get one power or the other, never both. The system refuses them by name, not by hiding a button.

03

What it does

IT DOES

Reads every payment as it happens

Accepts payments from the bank's systems, checks they are well formed,

IT DOES

Measures twenty-two things about each one

and stores them.

Not "is this a big payment" but "is this big *for this customer*" — compared against their own history. It also reads what happened before the money moved (a new phone, a changed PIN, a swapped SIM) and money coming in to the account.

IT DOES

Produces a probability

A trained model gives a number between zero and one. A separate set of written rules gives named warnings. A third piece combines them into a verdict.

IT DOES

Groups related payments into one case

Eleven suspicious transfers from one customer in ten minutes is one incident, not eleven separate jobs.

IT DOES

Tells the analyst what to do

A recommended action in one sentence, with the evidence behind it — and one keystroke to record the answer.

IT DOES

Writes down everything, permanently

Every decision, every setting change, every human action — in a record that cannot be edited or deleted afterwards.

And what it deliberately does not do

IT DOES NOT

Block or reverse any payment

It advises. A human acts. For two very precise warning signs it recommends a hold and flags the customer for a same-day call without waiting for an analyst, but whether the payment is held is still the bank's own decision.

IT DOES NOT

Connect to a real bank

No bank has agreed to work with us. Everything runs on our own infrastructure with invented data. The connection points to a bank's core system, the national identity register and other banks are built, with stand-

	ins in their place until a bank plugs in.
IT DOES NOT Store anyone's real account number Identifiers are scrambled one-way before anything is saved. Nothing stored can be turned back into a real account.	IT DOES NOT Claim to detect real fraud yet We can show the method is sound. We cannot show it works on real criminals, because we have never seen any. Section 7 is honest about this.

04

The three kinds of fraud we detect

We model three, chosen because they are structurally different from one another — which matters for testing, as section 6 explains.

NAME	WHAT ACTUALLY HAPPENS	WHAT IT LOOKS LIKE IN THE DATA
Account takeover	A thief gets hold of someone's login details, signs in as them, and empties the account.	A device nobody has seen before, a few small test payments, a pause, then a rush of transfers out.
Mule fan-out	Money is pushed out fast to many different accounts, to scatter it before anyone can freeze it.	One account paying several different destinations within minutes.
Card testing	A thief with a stolen card makes many tiny payments to find out whether it still works, then one large one.	A run of small payments, most of them refused, followed by a big one that goes through.

Where the data comes from

This is the part most likely to be challenged, so it is stated plainly. We wrote a program that invents the payments we train on. It creates fake customers with habits — when they are awake, how much they usually send, who they usually pay — and then has some of them robbed.

We did look for real data. There is no publicly available set of real Nigerian bank transactions with fraud marked on them. We checked three candidates and rejected all three, and the reasons are in the companion document. The published research says the same thing: authentic Nigerian fraud data is not available to researchers.

What we can do — and have done — is make our invented data match every real published statistic we can find. Nigeria's interbank settlement body publishes annual fraud figures. Our invented fraud rate is 0.305% against their reported 0.30%.

THE TRAP WE FELL INTO, AND FIXED

A program that invents fraud can accidentally make it too tidy. Ours did. At one point a single measurement could throw away 77% of all normal payments without losing a single fraudulent one — because our invented fraud always paid brand-new accounts and our invented normal customers almost never did.

That is not detection, it is a giveaway. We now run an automatic check for it on every rebuild, and the fix was to make our *normal* customers more realistic rather than making the fraud messier. Real people do pay new accounts, do pay fifteen people on salary morning, and do occasionally move large sums. Those people are exactly why fraud is hard to spot, and leaving them out had made the problem fake.

How we know whether it works

We test it on a crime it has never seen

The usual way to test is to hide some of your data, train on the rest, then check against what you hid. That answers "*can it recognise more of what it has already seen*". That is not the question that matters. Criminals change tactics, so next month's fraud is not in this month's data.

So we hide an entire *kind* of crime. Train on mule fan-out and card testing, then test on account takeover — a pattern the system has never encountered. Then rotate, and do it again for each.

The current, honest results

CRIME HIDDEN FROM TRAINING	TRAINED MODEL ALONE	MODEL PLUS WRITTEN RULES
Account takeover	0.92	0.99
Mule fan-out	0.36	0.91
Card testing	0.00	0.98
Average	0.43	0.96

Read that as: *of every ten fraud incidents of a type it had never seen, the trained model on its own caught about four; the full system caught more than nine*. These figures are measured with the system held to the team's chosen capacity of 75 alerts a day, on about 200 incidents of each kind. On 10 September, before the system could see warning events and incoming money, the full system averaged 0.89 at 120 alerts a day. Held to that capacity it now raises about three false alerts for each fraud, and flags nine-tenths of the money stolen.

The zero is the most useful number here. Shown card testing, a pattern completely absent from its training, the model caught one of 207. That is exactly what this kind of model does with something new, and it is the reason we keep a set of plainly written rules alongside it rather than relying on the model alone. The rules caught 202 of 207. Earlier, on a previous set of test data, they went from 185 to 192 of 194 once we fixed a bug that had been letting a "familiar payee" rule lower the risk on card payments, which have no payee at all.

The two halves divide the work. **The model carries account takeover:** on its own it catches 0.92 of it. A new rule that watches for warning signs before the money moves (a new phone and a changed PIN, then a payment somewhere new) now catches 0.84 by itself, and together they miss two in 210. That rule reads events our own program writes, so it is the figure most likely to fall on a real bank's data. **The rules are what catch card testing,** and most mule fan-out: on their own they catch 0.89 of it. A rule that follows the money in and then out again flags 85% of the money in mule rings, up from 64%.

Two trade-offs we chose, stated openly

- **The workload is capped, and the burst rule had to earn its place.**
The rule that watches for many payments in one hour was set to fire at five. With realistic customers — shop owners paying staff, traders paying suppliers — it alone raised half as many alerts again as the whole team can handle. Raising it to ten kept the queue workable but cut mule fan-out caught to about six in ten. So we changed what it looks at: it now fires on five or more payments in an hour *to a destination the bank first saw within the last day*. Traders pay people the bank has known for months, so the rule stays quiet for them; a mule network pays brand-new accounts. Against a fan-out pattern it has never seen, the system now catches about nine in ten, inside the same workload.

- **The probability is honest.** We fixed the model so that a stated 0.30 really means about three in ten. That changed how many frauds we catch not at all; it changed whether the team's workload can be predicted from the settings. It can now.

Three numbers we refuse to publish as proven

- **Accuracy.** Roughly three payments in a thousand are fraud. A system that answers "not fraud" to everything is 99.7% accurate and completely useless. A measure that rewards doing nothing is worse than no measure.
- **Any figure that reached 100%.** We had one — the system caught every mule fan-out. The check described in section 5 showed the rule was matching something our own program did every time, by construction. It has been withdrawn. Measured honestly, on a pattern hidden from training and with the workload capped, it now reads 0.91.
- **How many analysts we would save, as a bank figure.** Letting the system act on its two most precise warnings takes a third of the reviews off the desk (75 to 51 a day) with nothing missed, on our data. Right about 99 times in 100 on invented data is not a figure a bank should trust unread, so it waits for a trial alongside a real bank.

What we are entitled to claim

SAY EXACTLY THIS, AND NOTHING MORE

We cannot claim this system detects fraud. We can claim four things and prove each one: the method avoids the traps that make most fraud-detection results meaningless; the software is correct from end to end and runs on data we did not create; combining a trained model with written rules measurably beats the model alone on crimes it has never seen; and we have a written plan for what would turn our claim into a real one.

That is a stronger position than a big number we cannot defend, and it is the one an examiner who knows the field will respect.

What would settle it properly

1. **Test on real fraud labels.** One public set exists with genuine outcomes, and it is behind an account check that a person has to complete.
2. **Check our invented normal against published statistics.** Partly done; more to do.
3. **Run it alongside a real bank's system for a month or two, alerting nobody.** This is the only thing that truly answers the question, and it needs a bank.

In scope, out of scope

AREA	DECISION
Receiving payments, checking them, storing them	In. Built and working.
The twenty-two measurements, the trained model, the nine written rules, the verdict	In. Built and working.
Warning events before a payment, and money coming in	In. Built and working.
Nigerian refund and notification clocks, the 24-hour customer flag	In. Built and working; the clock values still to confirm against the regulator's circular.
Sharing that flag with other banks, identity by BVN	In. Built up to the connection; the live link needs a bank.
Letting the system act on its most precise warnings	In. Built; a bank should trial it before trusting it.
Grouping alerts into cases and the investigation workflow	In. Built and working.
The screen analysts work on	In. Built and working.
Sign-in, the four roles, the permanent record	In. Built and working.
Settings screens for the administrator	In. Built and working.
Making our invented customers realistic and checking for giveaways	In. Done for payments; the ten newer measurements still to check.
Testing each written rule individually against the data	In. Six of nine done.
Nigerian geography in the invented data	In. Not yet started.
Blocking or reversing payments	Out. Advisory only, permanently.

AREA	DECISION
Connecting to a bank's real systems	Out. Separate project.
Detecting networks of linked criminals	Out. Later version. Following one account's money in and out again is built.
Testing a new model on live traffic before switching to it	Out. Designed, not built.

09

Who does what

NAME	OWNS
Kanyinsola	Plan, documentation, test cases, final presentation.
Ire	Making the invented data realistic; testing each rule against the data; Nigerian geography. See the companion document — this is a change from the original split and the reasons are set out there.
Chidera	The service that receives and scores payments, the database, the permanent record, sign-in and roles.
Paul	The screens the analysts and the team lead use.

ONE DECISION STILL OUTSTANDING

The work on sign-in, roles and the permanent record grew beyond what was planned. Something else has to come out to pay for it. That trade has been flagged three times and still has not been made. **It needs deciding this week.**

Glossary

Alert

One payment the system thinks is worth a human look.

Case

A folder holding all the alerts about one customer from one incident.
The unit of work an analyst actually handles.

Model

A program that has learned patterns from examples, rather than following instructions somebody wrote. Ours outputs a probability.

Rule

An instruction somebody did write, in plain terms — for example, "this destination is on the sanctions list". Rules cannot be surprised, and cannot spot anything nobody thought of.

Probability

A number between zero and one. Ours is adjusted so it means what it says: 0.30 means roughly three in ten payments like this turn out to be fraud.

Recall

The share of real fraud that was caught. Nine out of ten incidents caught is a recall of 0.90.

Precision

The share of raised alerts that turned out to be real fraud. Low precision means analysts waste time.

Alert budget

How many alerts the team can actually review in a day. Three analysts reviewing forty each is 120. We set the system's sensitivity backwards from this number, rather than picking a threshold that sounds strict.

The simulator

Our program that invents fake customers and fake payments, including fake fraud.

Held-out crime

A kind of fraud deliberately removed from training so the system can be tested on something genuinely new to it.

Giveaway

A value our own program sets the same way every time it writes a fraud, which the model can then spot instead of learning real behaviour. Called a "generator switch" in the technical notes.

Risk Radar · scope and requirements · superseded documents: the original implementation plan and the earlier requirements document. Where they disagree with this one, this one is correct.
Companion: *What We Learned From Testing Both Approaches*.